

Topic: Defining Methods in JAVA

A method in Java represents an action on data or behavior of an object.

It is a collection of statements that are grouped together to perform an operation.

Method definition consists of a method header and a method body.

Syntax

```
modifier returnType nameOfMethod (Parameter List) {  
    // method body  
}
```

The syntax shown above includes –

- **modifier** – It defines the access type of the method and it is optional to use.
- **returnType** – Method may return a value.
- **nameOfMethod** – This is the method name. The method signature consists of the method name and the parameter list.
- **Parameter List** – The list of parameters, it is the type, order, and number of parameters of a method. These are optional, method may contain zero parameters.
- **method body** – The method body defines what the method does with the statements.

Example:

```
public static int sum(int a, int b) {  
    // body  
    return a+b;  
}
```

Here,

- **public static** – modifier
- **int** – return type
- **sum** – name of the method
- **int a, int b** – list of parameters

Method Calling

- For using a method, it should be called..
- When a program invokes a method, the program control gets transferred to the called method. This called method then returns control to the caller in two conditions, when
 - the return statement is executed.
 - it reaches the method ending closing brace.

Topic: Constructor in JAVA

- **Constructor** is a *special type of method* that is used to initialize the object.
- Java constructor is *invoked at the time of object creation*.
- Every class has a constructor, if we don't explicitly declare a constructor for any java class the compiler builds a default constructor for that class.

Rules for creating java constructor

There are basically two rules defined for the constructor.

- Constructor name must be same as its class name
- Constructor must have no explicit return type.

Types of java constructors

There are two types of constructors:

1. No-arg constructor
2. Parameterized constructor

Default Constructor

A constructor that have no parameter is known as default constructor.

Syntax of default constructor:

```
<class_name>() {  
    //body  
}
```

Example

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
class Bike{  
    Bike() {  
        System.out.println("Bike is created");  
    }  
    public static void main(String args[]){  
        Bike b=new Bike();  
    }  
}
```

Output:

Bike is created

Parameterized constructor

- A constructor that have parameters is known as parameterized constructor.
- Parameterized constructor is used to provide different values to the distinct objects.

Example

In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

```
class Student{
    int id;
    String name;
    Student(int i,String n){
        id = i;
        name = n;
    }
    void display(){
        System.out.println(id+" "+name);
    }
    public static void main(String args[]){
        Student s1 = new Student4(1234,"Aakarsh");
        Student s2 = new Student4(1279,"Bhavin");
        s1.display();
        s2.display();
    }
}
```

Output:

1234 Aakarsh

1279 Bhavin

Topic: Copy Constructor in Java

A copy constructor in a Java class is a constructor that **creates an object using another object of the same Java class.**

That's helpful when we want to copy a complex object that has several fields, or when we want to make a deep copy of an existing object.

To create a copy constructor, we can first declare a constructor that takes an object of the same type as a parameter:

Then, we copy each field of the input object into the new instance:

Example:

```
public class Employee {
    private int id;
    private String name;

    public Employee(Employee employee) {
        this.id = employee.id;
        this.name = employee.name;
    }
}
```

Topic: Constructor Overloading in Java

Constructor Overloading:

- Constructor overloading in Java refers to the practice of defining multiple constructors within a class, each with a different set of parameters.
- These parameters can vary in terms of their number, data types, and order.
- The purpose of constructor overloading is to provide multiple ways to initialize objects in a class, catering to different scenarios and requirements.

Rules for Constructor Overloading in Java

- **Constructor Name:** All constructors must have the same name as the class in which they are defined. This is a fundamental rule of constructors in Java.
- **Parameter Lists:** Overloaded constructors must have different parameter lists. The parameter lists can differ in terms of the number, order, and data types of parameters.
- **Return Type:** Constructors in Java do not have a return type, not even void. This rule applies to all constructors, including overloaded ones.
- **Access Modifiers:** Constructors can have any access modifier (public, protected, default, or private), but they cannot be marked as static as they are instance methods.

```
public class Car {
    // Fields (or properties) of the Car class
    private String model;
    private int year;
    private double price;

    // 1. Default Constructor
    public Car() {
        // Initializing with default values
        this.model = "Unknown Model";
        this.year = 0;
        this.price = 0.0;
    }

    // 2. Parameterized Constructor
    public Car(String model, int year, double price) {
```

```
        this.model = model;
        this.year = year;
        this.price = price;
    }

    // 3. Copy Constructor (Manually Implemented)
    public Car(Car other) {
        this.model = other.model;
        this.year = other.year;
        this.price = other.price;
    }

    // Method to display the car details
    public void displayInfo() {
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
        System.out.println("Price: $" + price);
    }

    // Main method to test constructors
    public static void main(String[] args) {
        // Using Default Constructor
        Car defaultCar = new Car();
        System.out.println("Default Car:");
        defaultCar.displayInfo();

        System.out.println();

        // Using Parameterized Constructor
        Car myCar = new Car("Tesla Model S", 2024, 79999.99);
        System.out.println("My Car:");
        myCar.displayInfo();

        System.out.println();

        // Using Copy Constructor
        Car copiedCar = new Car(myCar);
        System.out.println("Copied Car:");
        copiedCar.displayInfo();
    }
}
```

Output:

Default Car:
Model: Unknown Model
Year: 0
Price: \$0.0

My Car:
Model: Tesla Model S
Year: 2024

Price: \$79999.99

Copied Car:

Model: Tesla Model S

Year: 2024

Price: \$79999.99

Topic: Method Overloading in Java

- In Java, method overloading, is a part of the polymorphism concept.
- This feature allows a class to have more than one method with the same name, as long as the parameters are different.
- The difference in parameters can be in terms of the number of parameters or the type of parameters.
- This provides the flexibility to call a similar method for different types of data

Benefits of Using Method Overloading

- **Readability:** The Java overload method simplifies code by allowing multiple methods with the same name but different parameters, making it easier to understand and maintain.
- **Flexibility:** It provides flexibility by using the same method name for similar operations with varying inputs, reducing the need for multiple method names.
- **Conciseness:** Method overloading in Java reduces redundancy in code by consolidating similar functionalities into a single method name, improving code conciseness.
- **Ease of Use:** Developers can easily call overloaded methods without worrying about remembering multiple method names, enhancing code usability and developer experience.
- **Enhanced Efficiency:** Method overloading promotes efficient resource use and improves code efficiency by avoiding the creation of unnecessary methods with distinct names.

Different ways to achieve method overloading in Java:

1. Changing the Number of Parameters:

This involves defining multiple methods with the same name but a different number of parameters. Java distinguishes between these methods based on the number of parameters.

Example:

```
class Bank {  
    // Method to calculate balance for a single account  
    public double calculateBalance(double accountBalance) {  
        return accountBalance;  
    }  
  
    // Method to calculate balance for multiple accounts  
    public double calculateBalance(double account1Balance, double  
account2Balance) {  
        return account1Balance + account2Balance;  
    }  
}
```

```
}

class MethodOverloadingDemo{
    public static void main(String[] args){
        Bank bank = new Bank();
        // For single account
        double balance1 = bank.calculateBalance(1000.00);
        System.out.println("Balance for single account: " + balance1);
        // For multiple accounts
        double balance2 = bank.calculateBalance(1000.00, 2000.00);
        System.out.println("Total balance for multiple accounts: " +
            balance2);
    }
}
```

Output

```
Balance for single account: 1000.0
Total balance for multiple accounts: 3000.0
```

2. Changing Data Types of the Parameters

This involves defining multiple methods with the same name but different data types of parameters. Java selects the appropriate method based on the data types of parameters passed.

```
class Multiplication{
    int mult(int a,int b){    // method mult having 2 parameters
        return a*b;
    }
    //Method Overloading on datatype of parameters
    double mult(double a,double b){    // method mult overloaded
        return a*b;
    }
}

class Main{
    public static void main(String[] args){
        Multiplication obj = new Multiplication();
        System.out.println(obj.mult(10,9));
        System.out.println(obj.mult(10.5,9.8));
    }
}
```

Output:

```
90
102.9
```

Topic: 'this' keyword in Java

In Java, `this` keyword is a reference variable that refers to the current object—the object whose method or constructor is being called. It serves several purposes:

1. **Accessing Instance Variables:** We can use `this` to refer to instance variables when they are shadowed by method or constructor parameters.

```
public class MyClass {
```

```
        private int value;
        public MyClass(int value) {
            this.value = value;
        }
    }
```

2. Calling Other Constructors: Within a class, you can use `this()` to call another constructor in the same class. This is known as constructor chaining.

```
public class MyClass {
    private int value;
    private String name;

    public MyClass() {
        // Calls the constructor with two parameters
        this(1, "default");
    }

    public MyClass(int value, String name) {
        this.value = value;
        this.name = name;
    }
}
```

Topic: Nested classes in Java

In Java, we can define a class within another class. Such class is known as `nested class`. For example,

```
class OuterClass {
    // ...
    class NestedClass {
        // ...
    }
}
```

There are two types of nested classes you can create in Java.

- **Non-static nested class (inner class)**
- **Static nested class**

1. Non-Static Nested Class (Inner Class)

A non-static nested class is a class within another class. It has access to members of the enclosing class (outer class). It is commonly known as `inner class`.

Since the `inner class` exists within the outer class, you must instantiate the outer class first, in order to instantiate the inner class.

Example:

```
class CPU {
    double price;
    // nested class
    class Processor{

        // members of nested class
        double cores;
        String manufacturer;

        double getCache() {
            return 4.3;
        }
    }
}

// nested class
class RAM{

    // members of nested class
    double memory;
    String manufacturer;

    double getClockSpeed() {
        return 5.5;
    }
}

}

public class NestedClassDriver {
    public static void main(String[] args) {

        // create object of Outer class CPU
        CPU cpu = new CPU();

        // create an object of inner class Processor using outer class
        CPU.Processor processor = cpu.new Processor();

        // create an object of inner class RAM using outer class CPU
        CPU.RAM ram = cpu.new RAM();
        System.out.println("Processor Cache = " + processor.getCache());
        System.out.println("Ram Clock speed = " + ram.getClockSpeed());
    }
}
```

Output:

```
Processor Cache = 4.3
Ram Clock speed = 5.5
```

In the above program, there are two nested classes: *Processor* and *RAM* inside the outer class: *CPU*.

Inside the Main class,

- we first created an instance of an outer class *CPU* named *cpu*.
- Using the instance of the outer class, we then created objects of inner classes:

```
CPU.Processor processor = cpu.new Processor;
```

```
CPU.RAM ram = cpu.new RAM();
```

Note: We use the dot (.) operator to create an instance of the inner class using the outer class.

Accessing Members of Outer Class within Inner Class

We can access the members of the outer class by using **this** keyword.

```
class Car {
    String carName;
    String carType;

    // assign values using constructor
    public Car(String name, String type) {
        this.carName = name;
        this.carType = type;
    }

    // private method
    private String getCarName() {
        return this.carName;
    }
}

// inner class
class Engine {
    String engineType;
    void setEngine() {

        // Accessing the carType property of Car
        if(Car.this.carType.equals("Hatchback")){

            // Invoking method getCarName() of Car
            if(Car.this.getCarName().equals("Swift")) {
                this.engineType = "Smaller";
            } else {
                this.engineType = "Bigger";
            }

        }else{
            this.engineType = "Bigger";
        }
    }
    String getEngineType(){
        return this.engineType;
    }
}
```

```
public class NestedClassDriver1 {
    public static void main(String[] args) {

// create an object of the outer class Car
        Car car1 = new Car("XUV700", "MUV");

        // create an object of inner class using the outer
class
        Car.Engine engine = car1.new Engine();
        engine.setEngine();
        System.out.println("Engine Type for MUV= " +
engine.getEngineType());

        Car car2 = new Car("Swift", "Hatchback");
        Car.Engine c2engine = car2.new Engine();
        c2engine.setEngine();
        System.out.println("Engine Type for Hatchback = " +
c2engine.getEngineType());
    }
}
```

Output :

```
Engine Type for MUV= Bigger
Engine Type for Hatchback = Smaller
```

2. Static Nested Class

In Java, we can also define a **static** class inside another class. Such class is known as **static nested class**.

Unlike inner class, a static nested class cannot access the member variables of the outer class. It is because the **static nested class** doesn't require you to create an instance of the outer class.

```
OuterClass.NestedClass obj = new OuterClass.NestedClass();
```

Here, we are creating an object of the **static nested class** by simply using the class name of the outer class. Hence, the outer class cannot be referenced using `OuterClass.this`.

Example:

```
class MotherBoard {

    // static nested class
    static class USB{
        int usb2 = 2;
        int usb3 = 1;
        int getTotalPorts(){
            return usb2 + usb3;
        }
    } //USB
} //MotherBoard
```

```
public class NestedStaticClassDriver {
    public static void main(String[] args) {

        // create an object of the static nested class
        // using the name of the outer class
        MotherBoard.USB usb = new MotherBoard.USB();
        System.out.println("Total Ports = " + usb.getTotalPorts());
    }
}
```

Output:

Total Ports = 3

In the above program, we have created a static class named *USB* inside the class *MotherBoard*. Notice the line,

```
MotherBoard.USB usb = new MotherBoard.USB();
```

Here, we are creating an object of *USB* using the name of the outer class.

Accessing members of Outer class inside Static Inner Class

If you try to access the members of the outer class within a static nested class, we will get an error because we are not using the object of the outer class to create an object of the inner class. Hence, there is no reference to the outer class

Example:

```
class MotherBoard {
    String model;
    public MotherBoard(String model) {
        this.model = model;
    }

    // static nested class
    static class USB{
        int usb2 = 2;
        int usb3 = 1;
        int getTotalPorts(){
            // accessing the variable model of the outer class
            if(MotherBoard.this.model.equals("Lenovo")) {
                return 4;
            }
            else {
                return usb2 + usb3;
            }
        }
    }
}
```

```
public class StaticNestedClassDriver1 {
    public static void main(String[] args) {

        // create an object of the static nested class
        MotherBoard.USB usb = new MotherBoard.USB();
        System.out.println("Total Ports = " +
usb.getTotalPorts());
    }
}
```

Output:

```
StaticNestedClassDriver1.java:13: error: non-static variable
this cannot be referenced from a static context
        if (MotherBoard.this.model.equals("Lenovo")) {
                        ^
1 error
```

Topic: Anonymous inner class

Java anonymous inner class is an inner class without a name and for which only a single object is created. An anonymous inner class can be useful when making an instance of an object with certain "extras" such as overloading methods of a class or interface, without having to actually subclass a class.

In simple words, a class that has no name is known as an anonymous inner class in Java. It should be used if you have to override a method of class or interface. Java Anonymous inner class can be created in two ways:

1. Class (may be abstract or concrete).
2. Interface

1. Anonymous inner class example using class**TestAnonymousInner.java**

```
abstract class Person{
    abstract void eat();
}
class TestAnonymousInner{
    public static void main(String args[]){
        Person p=new Person(){
            void eat(){System.out.println("nice fruits");}
        };
        p.eat();
    }
}
```

Output:

```
nice fruits
```

Topic: Parameter Passing Mechanism in Java

In Java, the parameter passing mechanism is often described in terms of **pass-by-value** and pass-by reference

1. Pass-by-Value

When you pass primitive types (such as `int`, `float`, `char`, etc.) to a method, Java passes a **copy of the value**. This means that changes made to the parameter within the method do not affect the original variable outside the method.

```
class PrimitivePassingExample {
    public static void modifyPrimitive(int value) {
        value = 20; // This change is local to the method and
        // does not affect the originalValue
    }

    public static void main(String[] args) {
        int originalValue = 10;
        System.out.println("Original Value before method call:
" + originalValue);
        modifyPrimitive(originalValue);
        System.out.println("Original Value after method call:
" + originalValue);
    }
}
```

Output:

Original Value before method call: 10

Original Value after method call: 10

2. Pass-by-Reference

- In Java, reference types (like objects, arrays) are passed by reference.
- When a reference is passed to a method, a copy of the reference itself is created, but the copy points to the same object in memory as the original reference.
- Any changes made to the object through the reference within the method will affect the original object outside the method.

```
class MyObject {
    int value;

    MyObject(int value) {
        this.value = value;
    }
}
```

```

class ReferencePassingExample {
    public static void main(String[] args) {
        MyObject myObject = new MyObject(10);
        System.out.println("Original Object Value before
method call: " + myObject.value);
        modifyObject(myObject);
        System.out.println("Original Object Value after method
call: " + myObject.value);
    }
    public static void modifyObject(MyObject obj) {
        obj.value = 20; // Modifies the object's state, which
is visible outside the method
    }
}

```

Output:

Original Object Value before method call: 10

Original Object Value after method call: 20

In this example, modifyObject changes the state of the MyObject instance. The change is reflected outside the method because both the original reference and the parameter reference point to the same object.

Topic: Recursive Methods in Java

In Java, a method that calls itself is known as a **recursive method** and this process is known as **recursion**.

```

public static void main(String[] args) {
    ... ..
    recurse()
    ... ..
}

static void recurse() {
    ... ..
    recurse()
    ... ..
}

```

Working of Recursion inn java

Key Concepts of Recursion

1. **Base Case:** The condition under which the recursion stops. It prevents infinite recursion and eventual stack overflow.
2. **Recursive Case:** The part of the method where the recursion happens. It usually involves a call to the method itself with modified arguments.

Example: Factorial Calculation

The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n . It can be defined recursively as:

- $n! = 1$ if $n \leq 1$
- $n! = n \times (n - 1)!$ otherwise

Base Case: If n is less than or equal to 1, return 1.

Recursive Case: Multiply n by the factorial of $n - 1$.

```
class Factorial {  
    public static int factorial( int n ) {  
        if ( n <=1)    // Base case  
            return 1;  
        else  
            return n * factorial(n-1); // recursive case  
    }  
  
    public static void main(String[] args) {  
        int number = 4, result;  
        result = factorial(number);  
        System.out.println(number + " factorial = " + result);  
    }  
}
```

Output:

4 factorial = 24

Advantages:

- Simpler code for problems that are naturally recursive.
- Effective for divide-and-conquer problems.
- Manages state through the call stack.
- Can be more readable and intuitive.

Disadvantages:

- Performance overhead and risk of stack overflow.
- Higher memory usage due to stack frames.
- Harder to debug complex recursive calls.
- Potential inefficiency in cases of redundant computations.

Topic: Nesting of Methods in Java

- In Java, method nesting refers to the practice of calling one method from within another method.
- This is a common programming technique used to break down complex operations into simpler, manageable pieces, and to promote code reuse.

Key Concepts of Method Nesting

1. **Method Call:** When one method calls another method, the calling method is referred to as the **caller**, and the called method is referred to as the **callee**.
2. **Scope:** Methods can call other methods within the same class, or methods in other classes if they are accessible.
3. **Stack Behaviour:** Each method call creates a new stack frame, which allows methods to operate independently in terms of local variables and parameters.

Example:

```
public class MethodNestingExample {
    public static void main(String[] args) {
        // Call the main method
        methodA();
    }
    public static void methodA() {
        System.out.println("In methodA");
        // Call methodB from methodA
        methodB();
    }
    public static void methodB() {
        System.out.println("In methodB");
        // Call methodC from methodB
        methodC();
    }
    public static void methodC() {
        System.out.println("In methodC");
    }
}
```

Output:

```
In methodA
In methodB
In methodC
```

Example with Parameters and Return Values:

```
public class NestedMethodsExample {

    public static void main(String[] args) {
        int result = calculateSum(5, 10);
        System.out.println("The result is: " + result);
    }
}
```

```
public static int calculateSum(int a, int b) {  
    int sum = add(a, b);           // Calls add method  
    int square = square(sum);      // Calls square method  
    return square;  
}  
  
public static int add(int x, int y) {  
    return x + y; // Returns the sum of x and y  
}  
  
public static int square(int num) {  
    return num * num; // Returns the square of num  
}  
}
```

Output:

The result is: 225

Topic: Attribute (Variable) final and static in java

In Java, the `final` and `static` keywords can be used to modify class attributes (fields) to achieve specific behaviour.

final Keyword

The `final` keyword in Java is used to declare constants, immutable variables, and to prevent method overriding or inheritance. When applied to attributes, it indicates that the attribute's value cannot be changed once initialized.

Characteristics of final Attributes:

1. **Immutable:** Once a final attribute is assigned an attribute, its value cannot be modified. This ensures that the attribute remains constant throughout the lifetime of the program.
2. **Initialization:** A final attribute must be initialized either at the point of declaration or within the constructor of the class.
3. **Usage:** Commonly used for constants or values that should not change after their initial assignment.

static Keyword

The `static` keyword in Java indicates that a field or method belongs to the class rather than to instances of the class. Static members are shared among all instances of the class and can be accessed without creating an instance.

Characteristics of static Attributes:

1. **Class-Level Scope:** A `static` attribute is shared across all instances of the class. It exists once per class, not per instance.
2. **Initialization:** A `static` attribute is initialized when the class is first loaded into memory. It is not tied to any particular instance.
3. **Access:** `static` attributes can be accessed using the class name (e.g., `ClassName.attributeName`) or through an instance.

Combining final and static

When you combine `final` and `static`, you create a **constant** that is class-level and immutable. Such constants are typically used for values that should be consistent across all instances and cannot be changed.

Example:

```
public class ConstantsExample {
    // Static and final attribute (constant)
    public static final int MAX_USERS = 100;

    public static void main(String[] args) {
        // Accessing static final attribute using the class name
        System.out.println("Maximum users allowed: " +
            ConstantsExample.MAX_USERS);

        // Attempting to modify the constant would cause a
        compilation error
        ConstantsExample.MAX_USERS = 200;
    }
}
```

Output:

Compilation error

```
G:\RAJESH-LBRCE\OOPS\2024-25\notes>javac ConstantsExample.java
ConstantsExample.java:10: error: cannot assign a value to static final variable MAX_USERS
    ConstantsExample.MAX_USERS = 200;
                        ^
1 error
```